NURobotics Robotics Software Course

Week 5: SLAM & Mapping



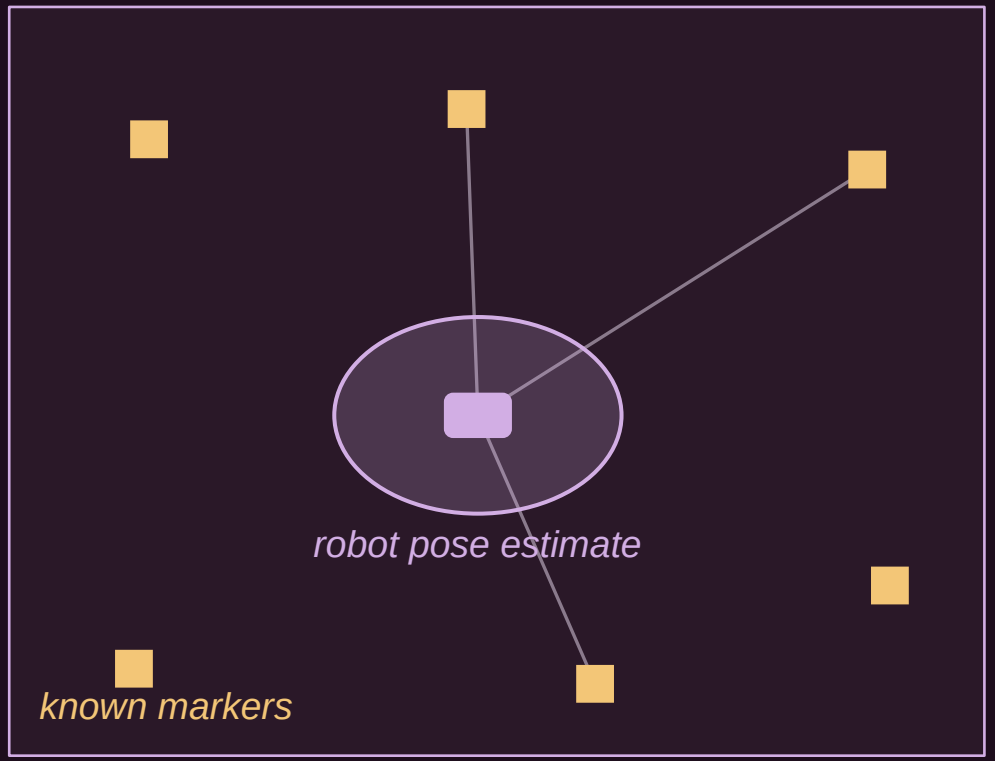
Simultaneous Localization and Mapping (SLAM)

Localization: Known Map

Known: the map (marker locations)

Estimate: the robot’s pose

Uncertainty: in the robot pose



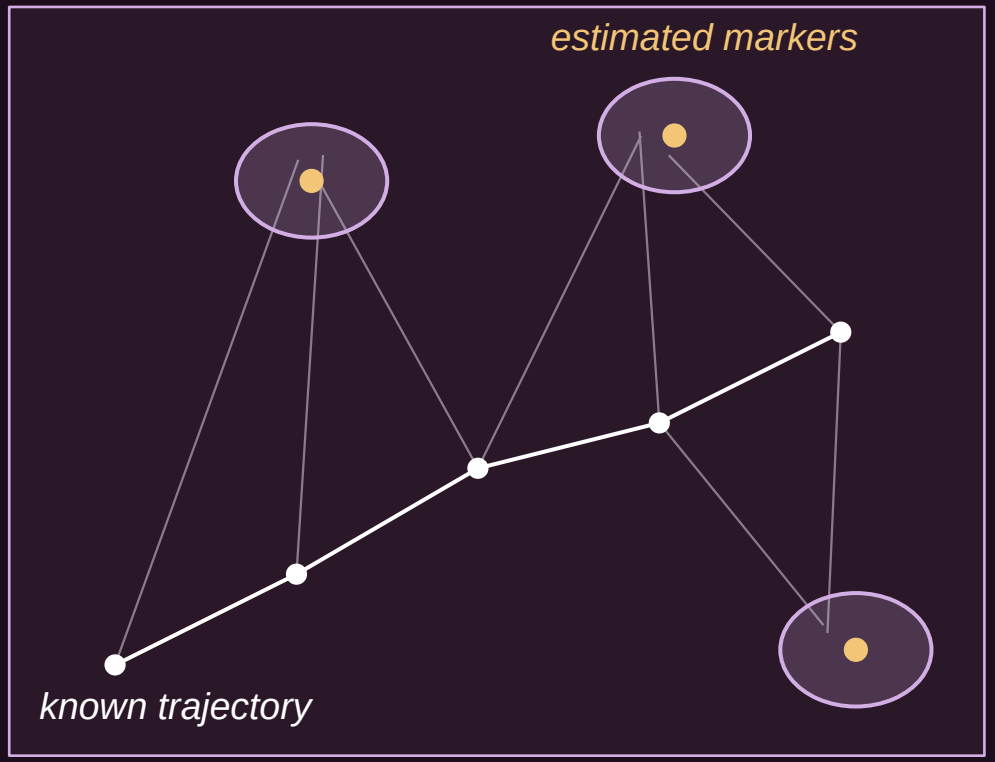
Markers are known; the robot’s pose is uncertain

Mapping: Known Pose

Known: where the robot was along its whole trajectory

Estimate: the marker locations

Uncertainty: in the marker measurements

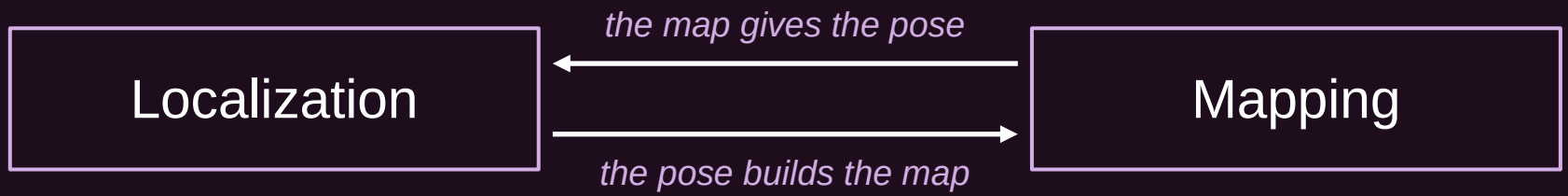


The trajectory is known; the markers are reconstructed from measurements

A Chicken-and-Egg Problem

Localization needs a map. Mapping needs localization.

In the real world, we often have neither

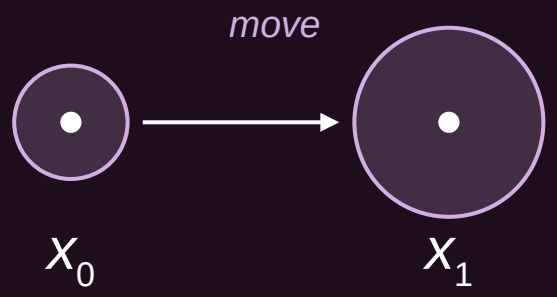


SLAM: solve both at the same time

Landmark SLAM

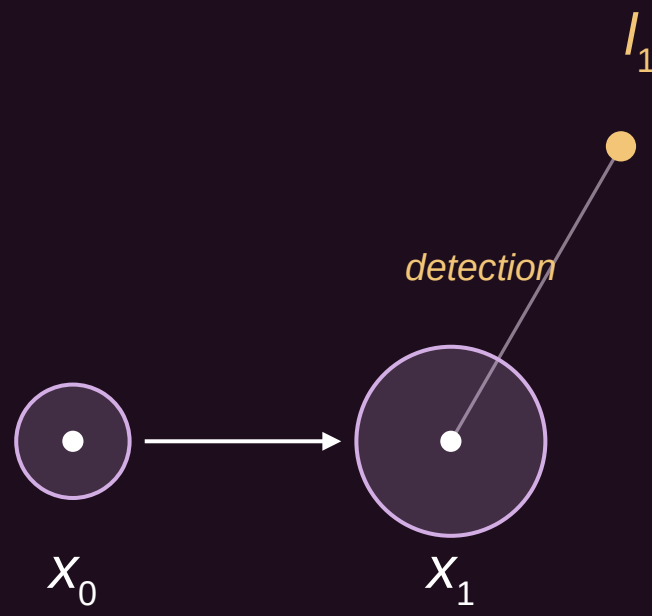
Landmark SLAM: Start and Move

Start with some uncertainty; moving increases it



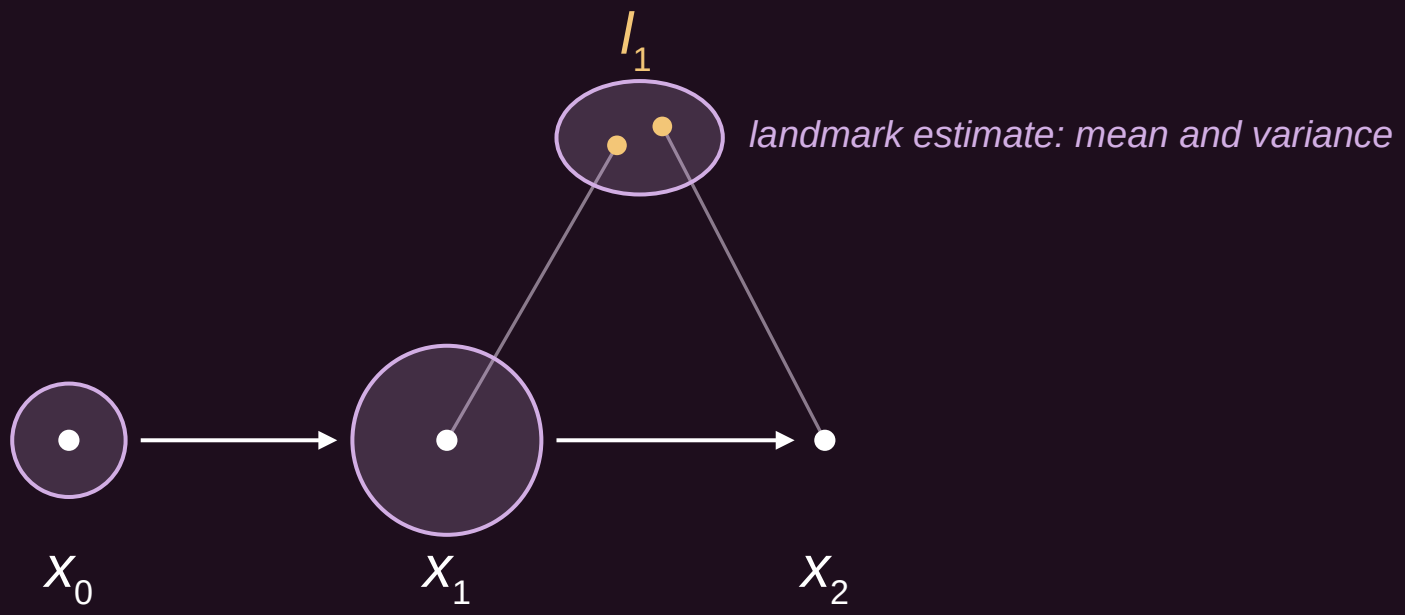
Landmark SLAM: Observe a Landmark

From x_1, the robot detects landmark l_1



Landmark SLAM: Observe It Again

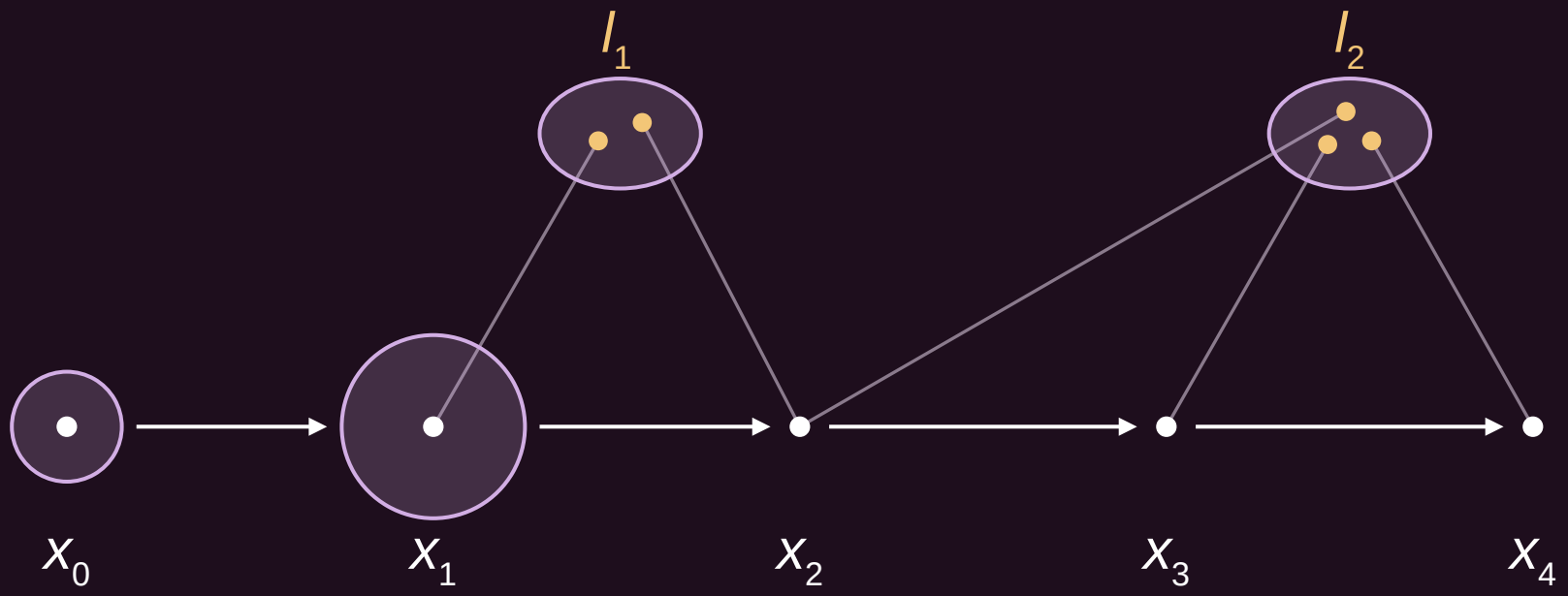
Seeing l_1 again from x_2 constrains x_1 and x_2



Position and measurement noise put each detection in a slightly different place

Landmark SLAM: Keep Driving

Every repeat detection adds another constraint



More Landmarks, More Constraints

Model how the robot moves and how accurate the sensors are

Constrain the poses and landmarks together → a decent map and decent localization

More landmarks → more constraints → a potentially better solution

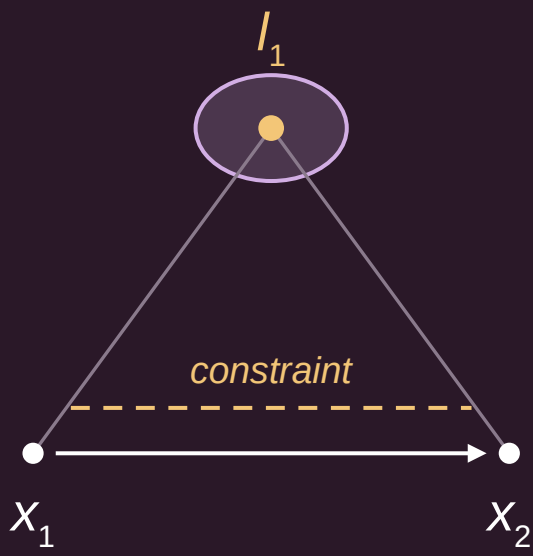
More landmarks → harder to compute each update

Landmark SLAM Issues

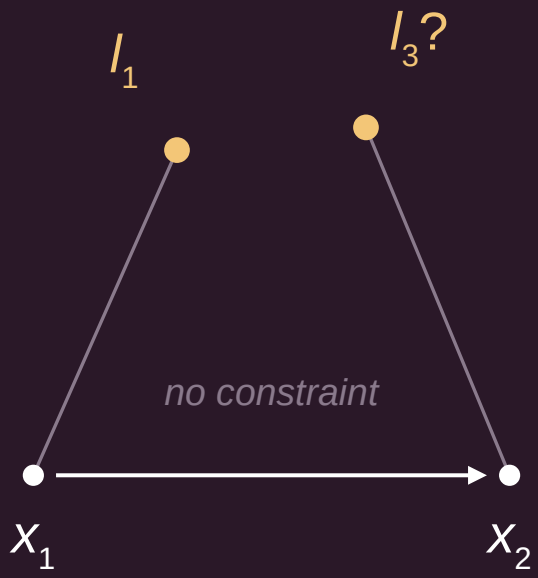
- Can we detect the same landmark again?
- Can we detect it from a different viewpoint? (e.g., the back of a sign)
- Can we detect it after not seeing it for a while?
  - The same hallway 10 minutes later: fairly easy
  - The same landscape: very hard, with lots of research

Issue: A Missed Re-detection

Recognized as the same landmark

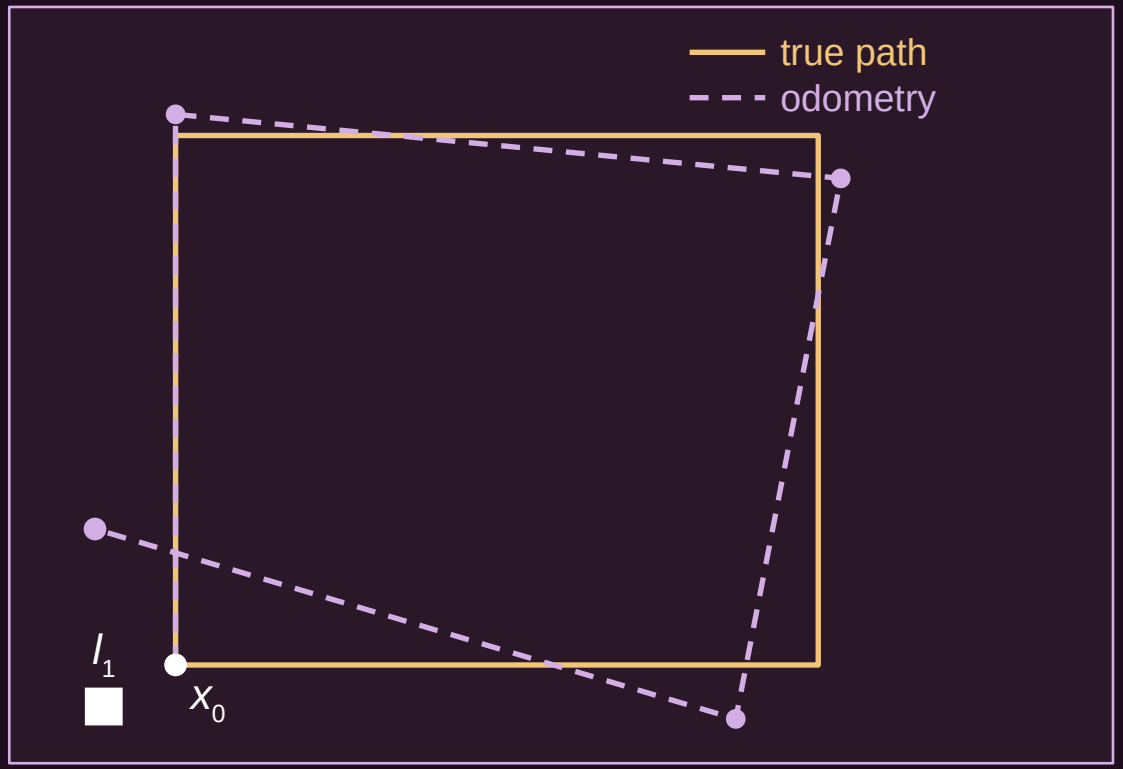


Seen as a new landmark



Without the match, x_1 and x_2 are no longer related

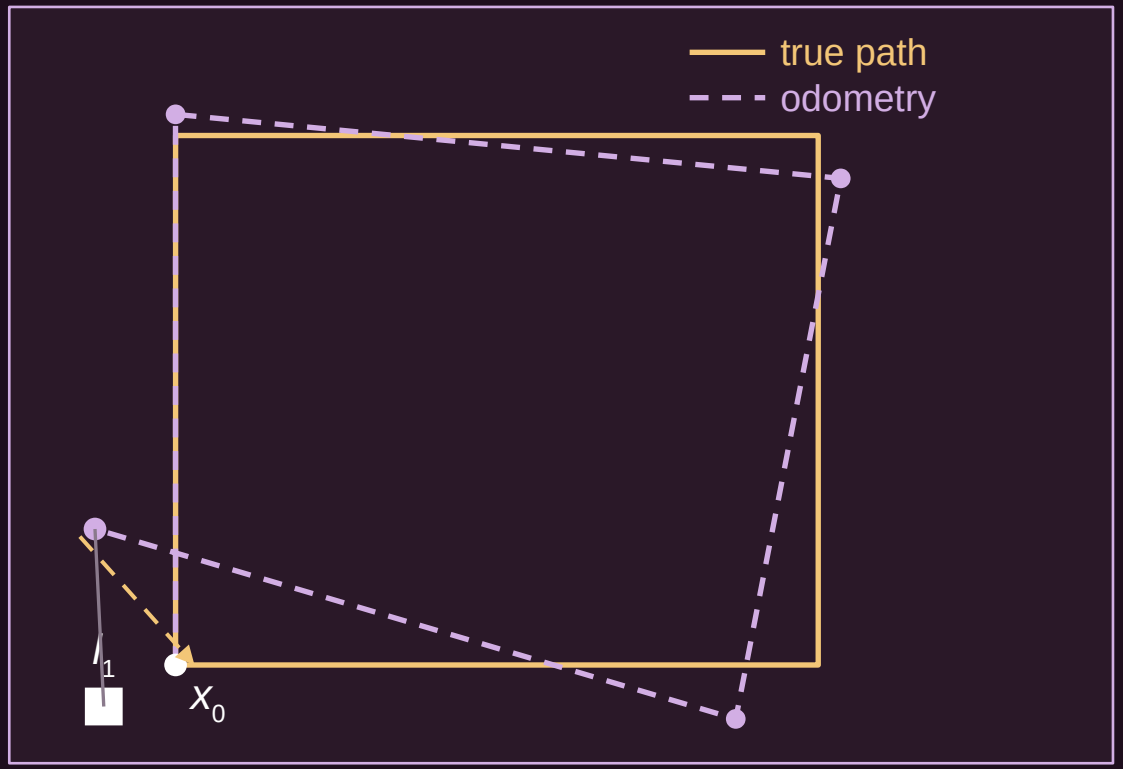
Loop Closure: Drift



Odometry alone slowly drifts away from the true path

The error builds up with every move

Loop Closure: Correction



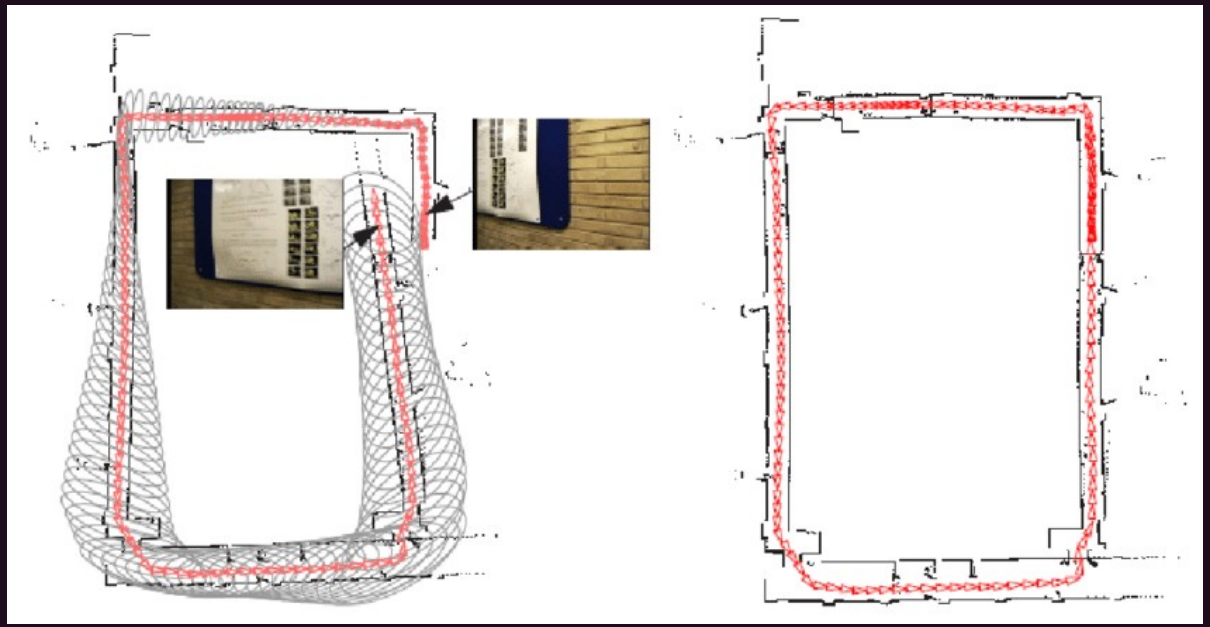
Seeing l_1 again relates the first and last poses, which corrects the whole path

Requires that we:

1. Detect the landmark
2. Recognize it as the same landmark
3. Relate the two poses, even if they aren’t identical

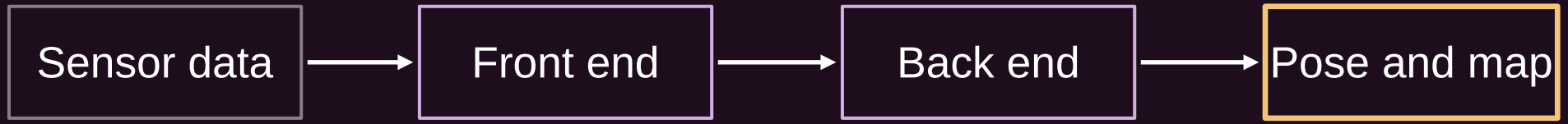
Loop Closure Example

Drive down a hallway, detect a sign, and return to it later



Front End and Back End

Front End and Back End



Feature extraction and tracking

Iterative closest point

Camera features

Turn features into pose and landmark updates

Filter-based (e.g., EKF)

Global optimization

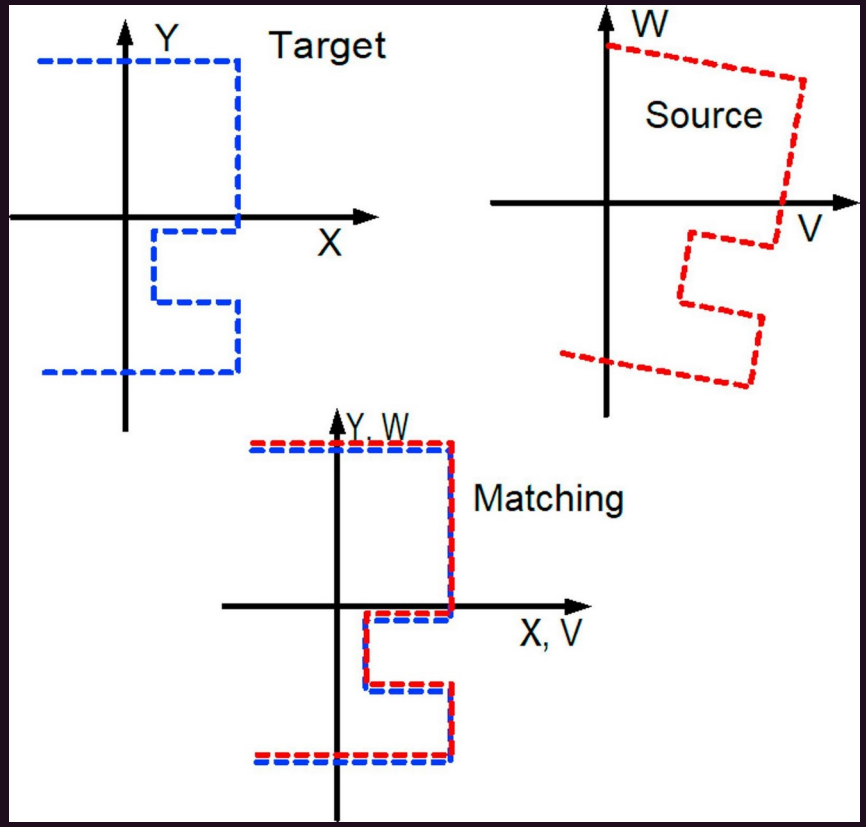
Machine learning

Front End: Iterative Closest Point

Target: our previous map estimate (we never have the ground-truth map)

Source: a new sensor scan

Align the scan to the map → a correction to the robot's pose



Front End: Camera Features

Detect distinctive points, like corners, that look similar from many angles

Common feature types: SIFT, FAST, ORB

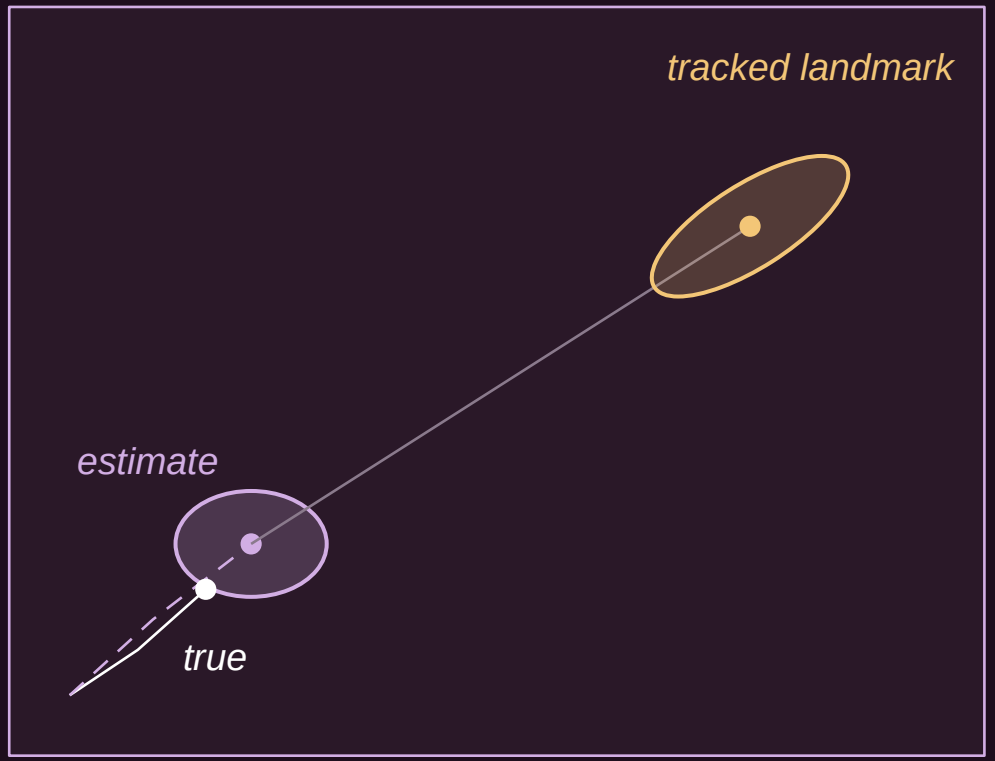


Back End: EKF SLAM

Tracks the current robot pose and landmark estimates, with their uncertainty

Landmark uncertainty shrinks as observations add up

No full history, so past errors are hard to correct

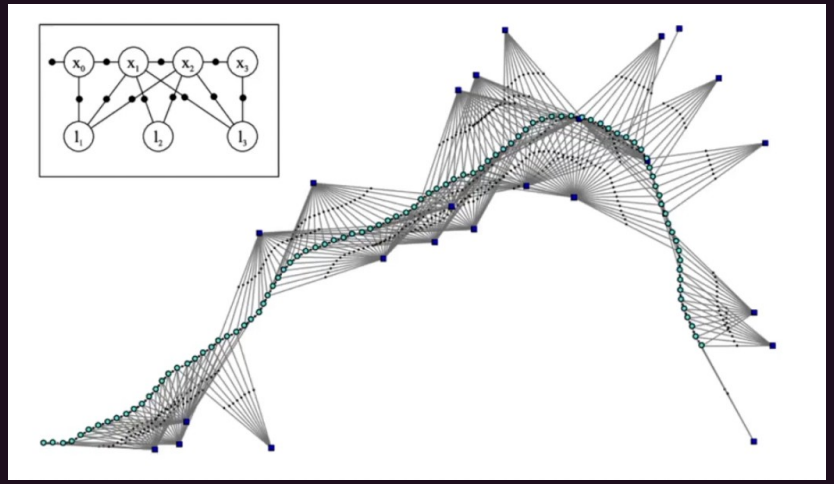


The ellipse’s shape comes from the sensor model

Back End: Graph-Based SLAM

Keeps the entire history of poses and sensor readings

Hard to solve in real time; incremental solvers help



Summary

SLAM builds a map while localizing in it: a chicken-and-egg problem

Solve incrementally: refine the position, then the map, and repeat

Front end: extract and track features

Back end: use features to constrain poses and landmarks

Next: different ways to represent maps

Introduction to Occupancy Grids

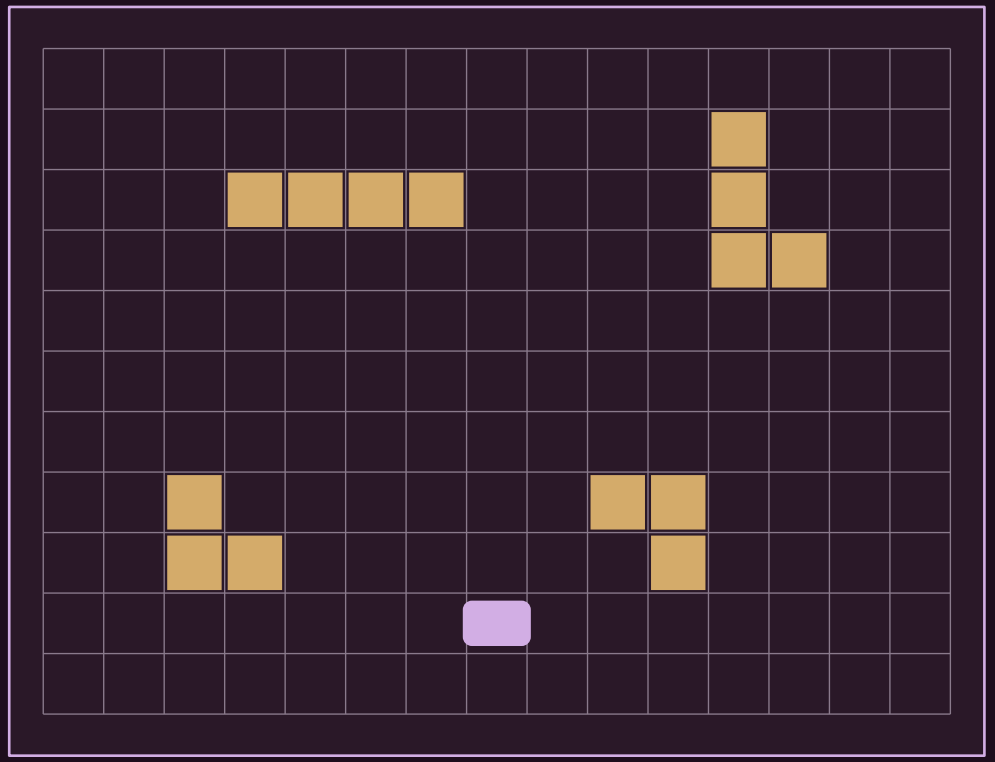
Occupancy Grid Mapping

Discretize the world into a grid

Each cell: is it occupied (an obstacle) or not?

The grid’s size is fixed before we start

The grid can be attached to the robot’s body frame and move with it



occupied cells

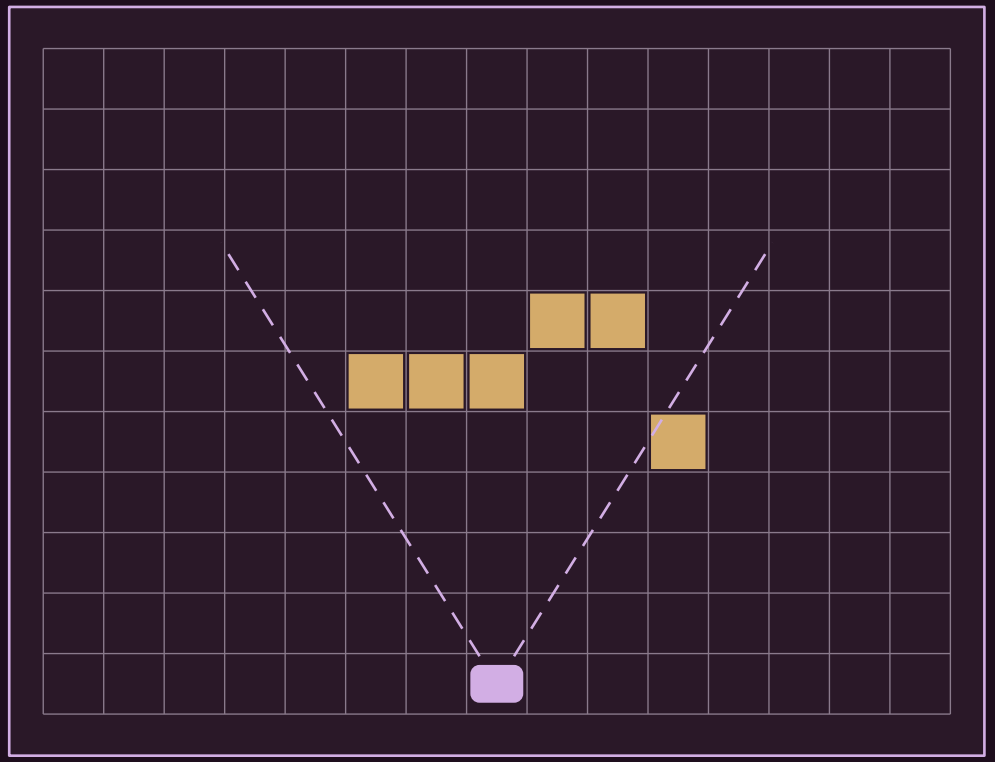
Can we drive over this cell or not?

What the Robot Sees

A camera detects red on the ground (as in week 2)

Project the detections onto the ground plane

Mark those cells as obstacles to avoid



detected red

The dashed lines show the camera’s field of view

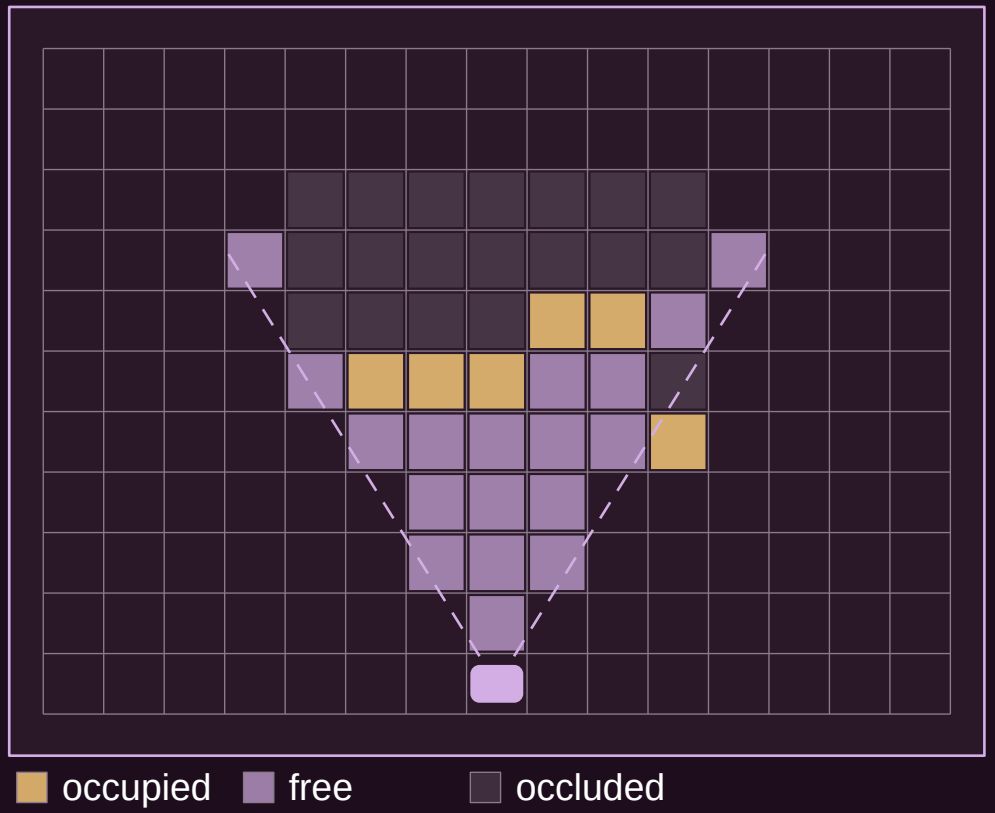
Occupied, Free and Occluded

Occupied: raise its probability

Free: lower its probability

Occluded: can’t see it, so leave it unchanged

You need all three for the map to work



Cells outside the view aren’t updated either

Each Cell Is a Random Variable

Each grid cell is a binary random variable: occupied or free

Only two events in the sample space, so they sum to one:

$P(\text{free}) = 1 - P(\text{occupied})$

Notation

$x \in \mathbb{R}^n$ – robot state

$u \in \mathbb{R}^n$ – robot control

$z \in \mathbb{R}^k$ – sensor reading

m_i – grid cell i

$P(m_i) \in [0, 1]$ — probability that cell i is occupied

Assumptions

1. A cell is either free or occupied
   - No third option
2. The world is static
   - No moving obstacles; handle them as best we can
3. Cells are independent of each other
   - Lets us estimate each cell on its own

Assumption: Independent Cells

One cell being occupied says nothing about its neighbors

Not always true, but it makes the problem much simpler



no information from its neighbor

The Best Map

$$M^* = \text{argmax}_m \, P(M \mid x_{1:t}, u_{1:t}, z_{1:t})$$

M: the entire map; * marks the best one

The map that makes the most sense given every state, control and sensor reading

Independence Gives a Product

$$\text{argmax}_m \, P(M \mid x_{1:t}, u_{1:t}, z_{1:t}) = \prod_i P(m_i \mid x_{1:t}, u_{1:t}, z_{1:t})$$

Estimate each cell on its own instead of the whole map at once

Summary

A grid of cells, each occupied or free

Update occupied, free and occluded cells differently

Assume binary cells, a static world and independent cells

The map becomes a product of per-cell estimates

Log Odds Derivation

Where We Left Off

The best map, one cell at a time:

$$P(m_i | x_{1:t}, u_{1:t}, z_{1:t})$$

Drop the controls and look at a single cell, written *m*:

$$P(m | z_{1:t}, x_{1:t})$$

Derivation: Separate the Newest Reading

Same first step as the particle filter: split z_t out of $z_{1:t}$

$$P(m|z_{1:t}, x_{1:t}) = P(m|z_t, z_{1:t-1}, x_{1:t})$$

Only the notation changes

Derivation: Bayes' Rule

Bayes' rule with an extra condition *c*:

$$P(x|y,c)=\frac{P(y|x,c)P(x|c)}{P(y|c)}$$

Applied with $x = m$, $y = z_t$ and $c = (z_{1:t-1}, x_{1:t})$:

$$P(m|z_t, z_{1:t-1}, x_{1:t})=\frac{P(z_t|m, z_{1:t-1}, x_{1:t})P(m|z_{1:t-1}, x_{1:t})}{P(z_t|z_{1:t-1}, x_{1:t})}$$

The Markov Property

The reading depends only on the current state and the cell:

$$P(z_t|m, z_{1:t-1}, x_{1:t}) = P(z_t|m, x_t)$$

Without z_t, the state x_t alone says nothing about the cell:

$$P(m|z_{1:t-1}, x_{1:t}) = P(m|z_{1:t-1}, x_{1:t-1})$$

Derivation: Apply the Markov Property

Substitute both simplifications:

$$P(m|z_{1:t}, x_{1:t}) = \frac{P(z_t|m, x_t)\,P(m|z_{1:t-1}, x_{1:t-1})}{P(z_t|z_{1:t-1}, x_{1:t})}$$

Derivation: Bayes' Rule Again

Flip the sensor term, keeping x_t as the condition:

$$P(z_t|m, x_t) = \frac{P(m|z_t, x_t)\, P(z_t|x_t)}{P(m|x_t)}$$

The state alone says nothing about the cell (independence):

$$P(m|x_t) = P(m)$$

Derivation: Put It Together

$$P(m|z_{1:t}, x_{1:t}) = \frac{P(m|z_t, x_t)\, p(z_t|x_t)\, P(m|z_{1:t-1}, x_{1:t-1})}{P(m)\, P(z_t|z_{1:t-1}, x_{1:t})}$$

Two terms are still hard to compute:

$P(z_t|x_t)$

a reading given only the state

$P(z_t|z_{1:t-1}, x_{1:t})$

a reading given earlier readings and states

The Complement: Not Occupied

$$P(m|z_{1:t}, x_{1:t}) = \frac{P(m|z_t, x_t)\, p(z_t|x_t)\, P(m|z_{1:t-1}, x_{1:t-1})}{P(m)\, P(z_t|z_{1:t-1}, x_{1:t})}$$

The cell is a binary random variable, so write the same equation for "not occupied" ($\neg m$):

$$P(\neg m|z_{1:t}, x_{1:t}) = \frac{P(\neg m|z_t, x_t)\, P(z_t|x_t)\, P(\neg m|z_{1:t-1}, x_{1:t-1})}{P(\neg m)\, P(z_t|z_{1:t-1}, x_{1:t})}$$

Derivation: Divide the Two

Divide the occupied equation by the free one; the hard terms cancel

$$\frac{P(m|z_{1:t}, x_{1:t})}{P(\neg m|z_{1:t}, x_{1:t})} = \frac{P(m|z_t, x_t)}{P(\neg m|z_t, x_t)} \frac{P(m|z_{1:t-1}, x_{1:t-1})}{P(\neg m|z_{1:t-1}, x_{1:t-1})} \frac{P(\neg m)}{P(m)}$$

Derivation: Use 1 − p

Replace each “not occupied” term with one minus “occupied”:

$$\frac{P(m|z_{1:t}, x_{1:t})}{1-P(m|z_{1:t}, x_{1:t})} = \underbrace{\frac{P(m|z_t, x_t)}{1-P(m|z_t, x_t)}}_{\textit{sensor update}} \underbrace{\frac{P(m|z_{1:t-1}, x_{1:t-1})}{1-P(m|z_{1:t-1}, x_{1:t-1})}}_{\textit{history}} \underbrace{\frac{1-P(m)}{P(m)}}_{\textit{map prior}}$$

The Prior

$P(m)$: how likely the cell is to be occupied before any readings

0.5 when unknown: occupied and free are equally likely

Close to 0 if the cell is very likely free; close to 1 if it’s likely an obstacle

Log Odds

Odds of a binary variable:

$$\text{odds}(x) = \frac{P(x)}{1 - P(x)}$$

$$P(x) = \frac{\text{odds}(x)}{1 + \text{odds}(x)}$$

Log odds:

$$l(x) = \log \frac{P(x)}{1 - P(x)}$$

$$P(x) = 1 - \frac{1}{1 + \exp(l(x))}$$

Our ratio is exactly the odds that the cell is occupied

Why Work in Log Space?

Products become sums, which are faster to compute

Better numerical stability on a computer

Stay in log-odds form and convert to a probability only when the map is needed

The Log-Odds Update

$$l_t = \underbrace{\log \frac{P(m|z_t, x_t)}{1 - P(m|z_t, x_t)}}_{\text{sensor update}} + \underbrace{l_{t-1}}_{\text{history}} - \underbrace{l_0}_{\text{prior}}$$

with the prior in log-odds form:

$$l_0 = \log \frac{P(m)}{1 - P(m)}$$

The Occupancy Grid Mapping Algorithm

Input: each cell's previous log odds, state x_t, reading z_t

for each cell m_i:

if m_i is in the perceptual field of z_t:

$$l_{t,i} = l_{t-1,i} + \text{ism}(m_i, x_t, z_t) - l_0$$

else:

$$l_{t,i} = l_{t-1,i}$$

return the updated log odds of every cell

ism: the inverse sensor model, i.e. the sensor update term in log-odds form

Summary

A true Bayesian estimate of each cell’s probability of being occupied

Log odds give a simple update: add the sensor term, subtract the prior

Fast, and every cell can be updated in parallel

An industry standard